CRYPTOGRAPHY AND NETWORK SECURITY
DA 1
23BCE0026

1. CEASER

AIM: To encrypt and decrypt a message using ceaser cypher

CODE:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
void process(char text[], int shift) {
for (int i = 0; text[i] != '\0'; i++) {
char c = text[i];
if (isalpha(c)) {
char base = isupper(c) ? 'A' : 'a';
text[i] = (char)((c - base + shift + 26) % 26 + base);
}
}
}
int main() {
char message[100];
int shift;
printf("Enter message: ");
fgets(message, sizeof(message), stdin);
message[strcspn(message, "\n")] = 0;
printf("Enter shift: ");
scanf("%d", &shift);
process(message, shift);
printf("23BCE0026");
printf("Encoded: %s\n", message);
process(message, -shift);
printf("Decoded: %s\n", message);
return 0;
}
```

OUTPUT:

```
Enter message: hello
Enter shift: 5
23BCE0026Encoded: mjqqt
Decoded: hello

Process returned 0 (0x0)   execution time : 49.526 s
Press ENTER to continue.
```

```
Enter message: hello
Enter shift: 23
23BCE0026Encoded: ebiil
Decoded: hello

Process returned 0 (0x0)   execution time : 7.408 s
Press ENTER to continue.
```

RESULT: Finally implemented Caeser cipher

2. PLAYFAIR

AIM: To encrypt and decrypt a message using playfair

CODE:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
void generateGrid(char key[], char grid[5][5]) {
int dict[26] = {0};
int k = 0;
for (int i = 0; i < strlen(key); i++) {
char c = toupper(key[i]);
if (c == 'J') c = 'I';
if (dict[c - 'A'] == 0 && isalpha(c)) {
grid[k / 5][k % 5] = c;
dict[c - 'A'] = 1;
k++;
}
}

for (int i = 0; i < 26; i++) {
if (i == ('J' - 'A')) continue;
if (dict[i] == 0) {
grid[k / 5][k % 5] = (char)(i + 'A');
dict[i] = 1;
k++;
}
}
}
void findPosition(char grid[5][5], char c, int *row, int *col) {
if (c == 'J') c = 'I';
for (int i = 0; i < 5; i++) {
for (int j = 0; j < 5; j++) {
if (grid[i][j] == c) {
*row = i; *col = j;
return;
}
}
}
}
void processText(char text[], char processed[]) {
int j = 0;
int len = strlen(text);
for (int i = 0; i < len; i++) {
if (!isalpha(text[i])) continue;
processed[j++] = toupper(text[i]);
if (i + 1 < len && toupper(text[i]) == toupper(text[i+1])) {
processed[j++] = 'X';
```

```c
if (j % 2 != 0) processed[j++] = 'X';
processed[j] = '\0';
}
void transform(char grid[5][5], char input[], char output[], int mode) {
int r1, c1, r2, c2;
int len = strlen(input);
int shift = (mode == 1) ? 1 : 4;
for (int i = 0; i < len; i += 2) {
findPosition(grid, input[i], &r1, &c1);
findPosition(grid, input[i+1], &r2, &c2);
if (r1 == r2) { // Same Row
output[i] = grid[r1][(c1 + shift) % 5];
output[i+1] = grid[r2][(c2 + shift) % 5];
} else if (c1 == c2) { // Same Column
output[i] = grid[(r1 + shift) % 5][c1];
output[i+1] = grid[(r2 + shift) % 5][c2];
} else { // Rectangle
output[i] = grid[r1][c2];
output[i+1] = grid[r2][c1];
}
}
output[len] = '\0';
}
int main() {
char key[100], text[100];
char processed[200] = {0}, encrypted[200] = {0}, decrypted[200] = {0};
char grid[5][5];
printf("Enter key: ");
scanf("%s", key);
generateGrid(key, grid);
printf("Enter message: ");
scanf("%s", text);
processText(text, processed);
transform(grid, processed, encrypted, 1);
transform(grid, encrypted, decrypted, 0);
printf("23BCE0026");
printf("\n--- PLAYFAIR CIPHER ---\n");
printf("1. Pre-processed: %s\n", processed);
printf("2. Encoded (Cipher): %s\n", encrypted);
printf("3. Decoded (Original): %s\n", decrypted);
return 0;
}
```

OUTPUT:

```
Enter key: COMPUTER
Enter message: COMMUNICATE
23BCE0026
--- PLAYFAIR CIPHER ---
1. Pre-processed: COMXMUNICATE
2. Encoded (Cipher): OMRMPCSGPTER
3. Decoded (Original): COMXMUNICATE

Process returned 0 (0x0)   execution time : 13.189 s
Press ENTER to continue.
```

```
Enter key: playfairexample
Enter message: hidethegoldinthetreestump
23BCE0026
--- PLAYFAIR CIPHER ---
1. Pre-processed: HIDETHEGOLDINTHETREXESTUMP
2. Encoded (Cipher): BMODZBXDNABEKUDMUIXMMOUVIF
3. Decoded (Original): HIDETHEGOLDINTHETREXESTUMP

Process returned 0 (0x0)   execution time : 30.259 s
Press ENTER to continue.
```

RESULT: Finally implemented Playfair cipher

3. VIGNERE

AIM: To encrypt and decrypt a message using vignere

CODE:

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>
void vigenere(char text[], char key[], int encrypt) {
int textLen = strlen(text);
int keyLen = strlen(key);
for (int i = 0, j = 0; i < textLen; i++) {
if (isalpha(text[i])) {
int shift = toupper(key[j % keyLen]) - 'A';
char base = isupper(text[i]) ? 'A' : 'a';
if (encrypt)
text[i] = (text[i] - base + shift) % 26 + base;
else
text[i] = (text[i] - base - shift + 26) % 26 + base;
j++;
}
}
}
int main() {
char msg[100], key[100];
printf("Enter message: ");
fgets(msg, sizeof(msg), stdin);
msg[strcspn(msg, "\n")] = 0;
printf("Enter keyword: ");
scanf("%s", key);
vigenere(msg, key, 1);
printf("23BCE0026");

printf("Encrypted: %s\n", msg);
vigenere(msg, key, 0);
printf("Decoded: %s\n", msg);
return 0;
}
```

OUTPUT:

```
Enter message: HELLO
Enter keyword: HI
23BCE0026Encrypted: OMSTV
Decoded: HELLO

Process returned 0 (0x0)   execution time : 6.166 s
Press ENTER to continue.
```

RESULT: Finally implemented Vignere cipher

4. VERNAM

AIM: To encrypt and decrypt a message using vernam

CODE:

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>
void encrypt(char* msg, char* key, int len) {
printf("Encoded: ");
for (int i = 0; i < len; i++) {
// Map A-Z to 0-25
int m = toupper(msg[i]) - 'A';
int k = toupper(key[i]) - 'A';
// Vernam Addition Modulo 26
int cipherNum = (m + k) % 26;
printf("%c", cipherNum + 'A');
// Store for decryption demo
msg[i] = cipherNum + 'A';
}
printf("\n");
}
void decrypt(char* cipher, char* key, int len) {
printf("Decoded: ");
for (int i = 0; i < len; i++) {
int c = toupper(cipher[i]) - 'A';
int k = toupper(key[i]) - 'A';
// Vernam Subtraction Modulo 26 (add 26 to handle negatives)
int plainNum = (c - k + 26) % 26;
printf("%c", plainNum + 'A');
}
printf("\n");
}
int main() {
char message[100], key[100];
printf("Enter Message (Alphabets only): ");
scanf("%s", message);
printf("Enter Key (Alphabets only): ");
scanf("%s", key);
int mLen = strlen(message);
int kLen = strlen(key);
if (kLen < mLen) {
printf("Error: Key length must be >= Message length\n");
return 1;
}
// The key is naturally "cut" because we only loop up to mLen
printf("23BCE0026");
printf("\n--- Results ---\n");
```

```c
printf("Enter Key (Alphabets only): ");
scanf("%s", key);
int mLen = strlen(message);
int kLen = strlen(key);
if (kLen < mLen) {
printf("Error: Key length must be >= Message length\n");
return 1;
}
// The key is naturally "cut" because we only loop up to mLen
printf("23BCE0026");
printf("\n--- Results ---\n");
encrypt(message, key, mLen);
decrypt(message, key, mLen);
return 0;
}
```

OUTPUT:

```
Enter Message (Alphabets only): SHREYA
Enter Key (Alphabets only): CRYPTOGRAPHY
23BCE0026
--- Results ---
Encoded: UYPTRO
Decoded: SHREYA

Process returned 0 (0x0)   execution time : 11.403 s
Press ENTER to continue.
```

RESULT: Finally implemented Vernam cipher

5. HILL CIPHER

AIM: To encrypt and decrypt a message using hill cipher

CODE:

```
#include<stdio.h>
#include<string.h>

int mod26(int x) {
return (x % 26 + 26) % 26;
}

int modInverse(int a) {
a = mod26(a);
for (int i = 1; i < 26; i++)
if ((a * i) % 26 == 1)
return i;
return -1;
}
int main() {
char text[] = "ACT";
char keyStr[] = "GYBNQKURP";
char cipher[4], decrypted[4];
int key[3][3], invKey[3][3];
int i, j, k = 0;
printf("23BCE0026\n");
for (i = 0; i < 3; i++)
for (j = 0; j < 3; j++)
key[i][j] = keyStr[k++] - 'A';
```

```
for (i = 0; i < 3; i++)
for (j = 0; j < 3; j++)
key[i][j] = keyStr[k++] - 'A';

int p[3];
for (i = 0; i < 3; i++)
p[i] = text[i] - 'A';
for (i = 0; i < 3; i++) {
int sum = 0;
for (j = 0; j < 3; j++)
sum += key[i][j] * p[j];
cipher[i] = mod26(sum) + 'A';
}
cipher[3] = '\0';

int det =
key[0][0]*(key[1][1]*key[2][2] - key[1][2]*key[2][1]) -
key[0][1]*(key[1][0]*key[2][2] - key[1][2]*key[2][0]) +
key[0][2]*(key[1][0]*key[2][1] - key[1][1]*key[2][0]);
det = mod26(det);
int invDet = modInverse(det);

invKey[0][0] = mod26((key[1][1]*key[2][2] - key[1][2]*key[2][1]) * invDet);
invKey[0][1] = mod26(-(key[0][1]*key[2][2] - key[0][2]*key[2][1]) * invDet);
invKey[0][2] = mod26((key[0][1]*key[1][2] - key[0][2]*key[1][1]) * invDet);
invKey[1][0] = mod26(-(key[1][0]*key[2][2] - key[1][2]*key[2][0]) * invDet);
invKey[1][1] = mod26((key[0][0]*key[2][2] - key[0][2]*key[2][0]) * invDet);
invKey[1][2] = mod26(-(key[0][0]*key[1][2] - key[0][2]*key[1][0]) * invDet);
invKey[2][0] = mod26((key[1][0]*key[2][1] - key[1][1]*key[2][0]) * invDet);
invKey[2][1] = mod26(-(key[0][0]*key[2][1] - key[0][1]*key[2][0]) * invDet);
invKey[2][2] = mod26((key[0][0]*key[1][1] - key[0][1]*key[1][0]) * invDet);
```

```
int c[3];
for (i = 0; i < 3; i++)
c[i] = cipher[i] - 'A';
for (i = 0; i < 3; i++) {
int sum = 0;
for (j = 0; j < 3; j++)
sum += invKey[i][j] * c[j];
decrypted[i] = mod26(sum) + 'A';
}
decrypted[3] = '\0';
printf("Encrypted Text : %s\n", cipher);
printf("Decrypted Text : %s\n", decrypted);
return 0;
}
```

OUTPUT:

```
23BCE0026
Encrypted Text : POH
Decrypted Text : ACT

Process returned 0 (0x0)   execution time : 0.005 s
Press ENTER to continue.
```

RESULT: Finally implemented Hill cipher